

Combinational Circuits: Adders & Subtractors

Half & full adder · half & full subtractor · ripple carry · carry look-ahead · adder-subtractor · BCD adder

QUICK TRICK

Quick tricks are in green boxes. The big trick for this chapter: **Sum = XOR of the inputs. Carry = 1 when two or more inputs are 1.**

9.1 What is a combinational circuit?

● LEVEL 1 · BEGINNER

What is it? A **combinational circuit** gives its output using **only the inputs it has right now**. It has **no memory** and **no clock**. If the inputs change, the output changes (after a tiny delay).

Analogy. A calculator that shows the result of what you are pressing right now, and forgets everything when you stop.

Combinational:	Inputs ---> [logic gates only] ---> Outputs	(no memory)
Sequential :	Inputs ---> [gates + memory] ---> Outputs	(Chapter 11)

Examples: adder, subtractor, multiplexer, demultiplexer, encoder, decoder, comparator. In this chapter we study the adders and subtractors.

EASY MEMORY

Combinational = **no memory**, output depends on **present inputs only**. Sequential = has memory (flip-flops).

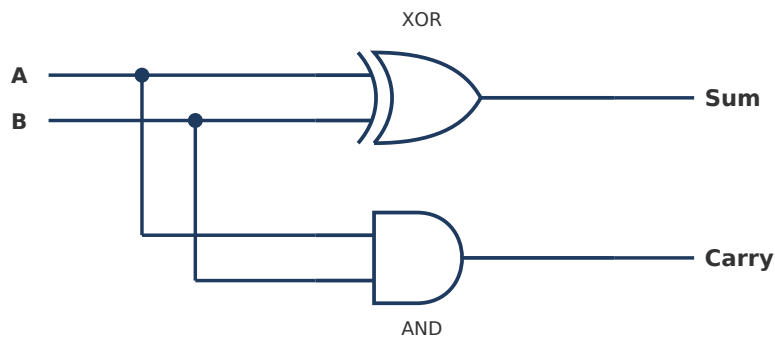
9.2 Half adder

● LEVEL 1 · BEGINNER

Problem. Add two single bits A and B. **Idea.** The answer can be 0, 1 or 2. Two (binary 10) needs **two output bits**: a **Sum** bit and a **Carry** bit.

A	B	Sum	Carry
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

Sum	= A XOR B	(A (+) B)
Carry	= A . B	



QUICK TRICK

Half adder = 1 XOR gate + 1 AND gate. Sum is 1 when the inputs are different. Carry is 1 only when 1 + 1.

COMMON MISTAKE

It is called “half” because it cannot accept a **carry coming in** from a previous column. So it can only be used for the very first (rightmost) column.

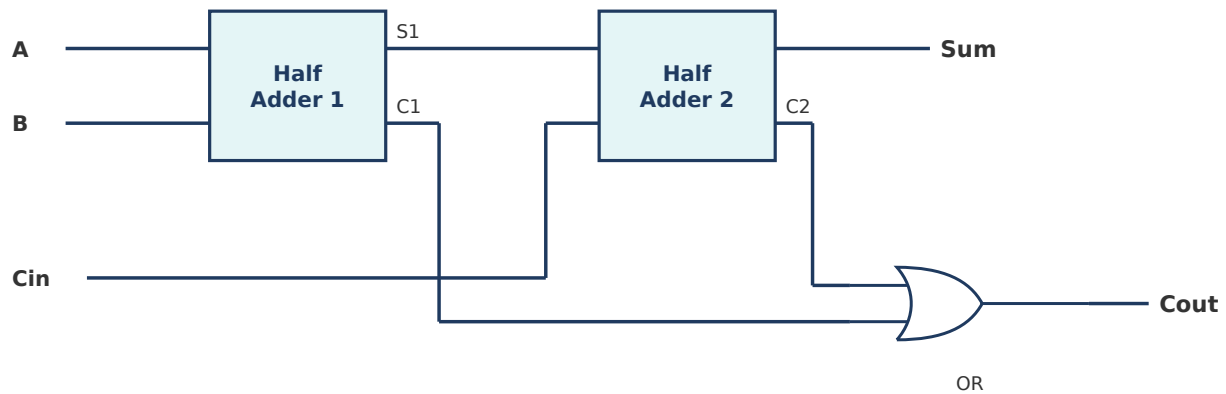
9.3 Full adder

● LEVEL 2 • EXAM

Problem. In a multi-bit addition, each column also receives the **carry from the column on the right**. So we must add **three bits**: A, B and the carry-in (C_{in}). **Idea.** Same two outputs: Sum and carry-out (C_{out}).

A	B	C_{in}	Sum	C_{out}
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

$$\begin{aligned}
 \text{Sum} &= A \text{ XOR } B \text{ XOR } C_{in} &= \text{SIGMA } m(1, 2, 4, 7) \\
 \text{Cout} &= AB + C_{in} (A \text{ XOR } B) &= AB + B.C_{in} + A.C_{in} &= \text{SIGMA } m(3, 5, 6, 7)
 \end{aligned}$$



How it works. Half adder 1 adds A and B (giving S1 and C1). Half adder 2 adds S1 and Cin (giving Sum and C2). A carry comes out if **either** half adder made a carry, so C1 and C2 go into an OR gate.

QUICK TRICK

Odd/majority trick. Sum = 1 when an **ODD** number of inputs (1 or 3) are 1. Cout = 1 when **TWO OR MORE** inputs are 1 (majority). Count the 1s: 0 → Sum 0, Cout 0 | 1 → Sum 1, Cout 0 | 2 → Sum 0, Cout 1 | 3 → Sum 1, Cout 1. (It is just the binary value of the count of 1s: Cout Sum.)

GATE POINT

Gate counts: half adder = 2 gates (XOR, AND). Full adder = 2 half adders + 1 OR = **5 gates** (2 XOR, 2 AND, 1 OR). Using only NAND gates: half adder = **5** NANDs, full adder = **9** NANDs.

9.4 Half subtractor and full subtractor

● LEVEL 2 • EXAM

Half subtractor subtracts B from A ($A - B$). Outputs: **Difference** and **Borrow** (borrow is 1 when A is smaller than B).

A	B	Difference	Borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

$$\text{Difference} = A \text{ XOR } B$$

$$\text{Borrow} = A' \cdot B$$

Full subtractor also subtracts a borrow that came from the previous column (Bin): $A - B - \text{Bin}$.

A	B	Bin	Diff	Bout
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$\begin{aligned} \text{Diff} &= A \oplus B \oplus \text{Bin} &= \text{SIGMA } m(1, 2, 4, 7) \\ \text{Bout} &= A'B + A'\text{Bin} + B\text{Bin} &= \text{SIGMA } m(1, 2, 3, 7) \end{aligned}$$

QUICK TRICK

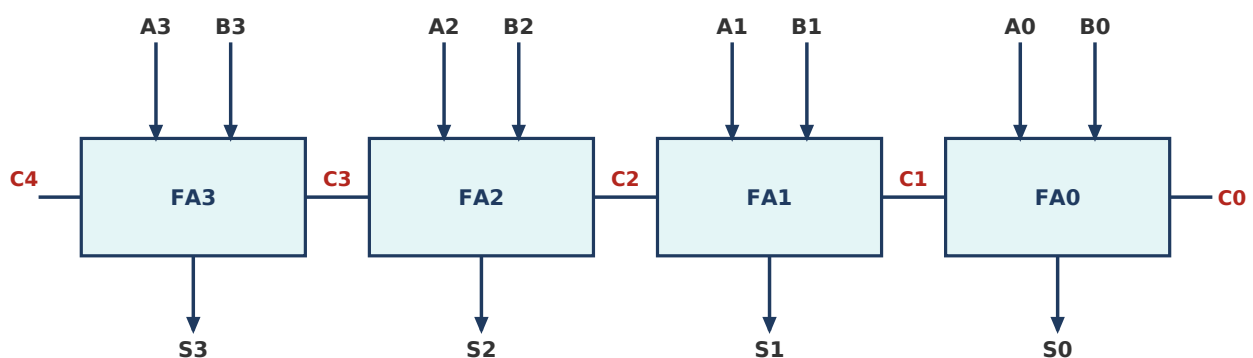
Subtractor = adder with ONE change. Difference is the **same XOR** as the Sum. The only change is in the borrow: **Borrow = A'B** (A is complemented) instead of Carry = AB. Full subtractor: Bout = 1 when $A - B - \text{Bin}$ is negative.

	Half adder	Half subtractor
Sum / Difference	$A \oplus B$	$A \oplus B$
Carry / Borrow	$A \cdot B$	$A' \cdot B$

9.5 Parallel (ripple carry) adder

● LEVEL 2 · EXAM

Idea. To add two n-bit numbers, join **n full adders** in a row. The carry-out of each adder goes to the carry-in of the next one.



Problem: carry propagation delay. Adder FA1 cannot finish until it gets C1 from FA0. FA2 must wait for FA1, and so on. The carry has to **ripple** through all adders, one after another. So the bigger the number of bits, the slower the adder.

QUICK TRICK

Delay trick: total delay $\approx n \times (\text{carry delay of one full adder})$. Each full adder carry takes about **2 gate delays** (AND, then OR). So an n-bit ripple adder is about **2n gate delays**. Example: 16 bits $\rightarrow$ about 32 gate delays.

EASY MEMORY

n-bit ripple carry adder needs **n full adders**. (The first one can be a half adder if there is no Cin.)

9.6 Carry look-ahead adder (CLA)

● LEVEL 3 • GATE

Problem. The ripple adder is slow because every stage waits for the carry. **Idea.** Work out **all the carries at the same time** directly from A, B and C0, without waiting.

For every bit i we define two simple signals:

Name	Formula	Meaning in simple words
Generate G_i	$A_i \cdot B_i$	This stage makes a carry by itself (1 + 1)
Propagate P_i	$A_i \oplus B_i$	This stage passes on a carry that comes in (exactly one input is 1)

$$C(i+1) = G(i) + P(i) \cdot C(i)$$

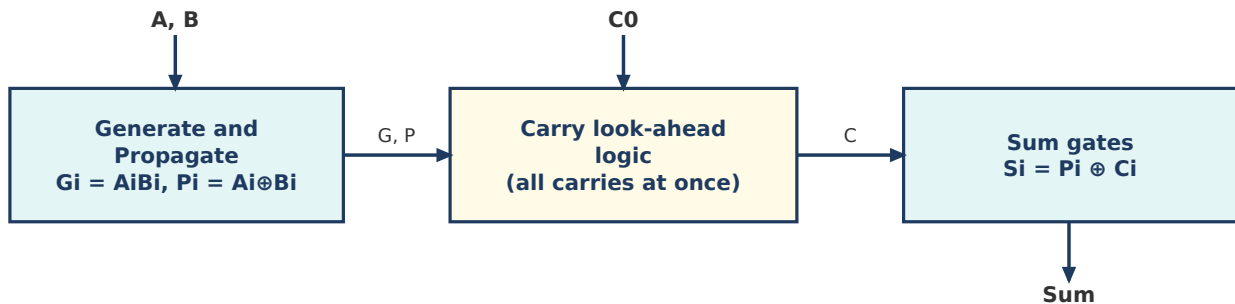
$$Sum(i) = P(i) \text{ XOR } C(i)$$

$$C1 = G0 + P0 \cdot C0$$

$$C2 = G1 + P1 \cdot G0 + P1 \cdot P0 \cdot C0$$

$$C3 = G2 + P2 \cdot G1 + P2 \cdot P1 \cdot G0 + P2 \cdot P1 \cdot P0 \cdot C0$$

$$C4 = G3 + P3 \cdot G2 + P3 \cdot P2 \cdot G1 + P3 \cdot P2 \cdot P1 \cdot G0 + P3 \cdot P2 \cdot P1 \cdot P0 \cdot C0$$



QUICK TRICK

Read $C(i+1) = G(i) + P(i) \cdot C(i)$ in words: "a carry goes out if this stage **generates** one, **or** if it **passes on** a carry that came in."

Example (Level 3): 0110 + 0101, C0 = 0

Bits (A3A2A1A0 = 0110, B3B2B1B0 = 0101): G0 = 0, P0 = 1 | G1 = 0, P1 = 1 | G2 = 1, P2 = 0 | G3 = 0, P3 = 0

$$C1 = G0 + P0 \cdot C0 = 0. C2 = G1 + P1 \cdot C1 = 0. C3 = G2 + P2 \cdot C2 = 1. C4 = G3 + P3 \cdot C3 = 0.$$

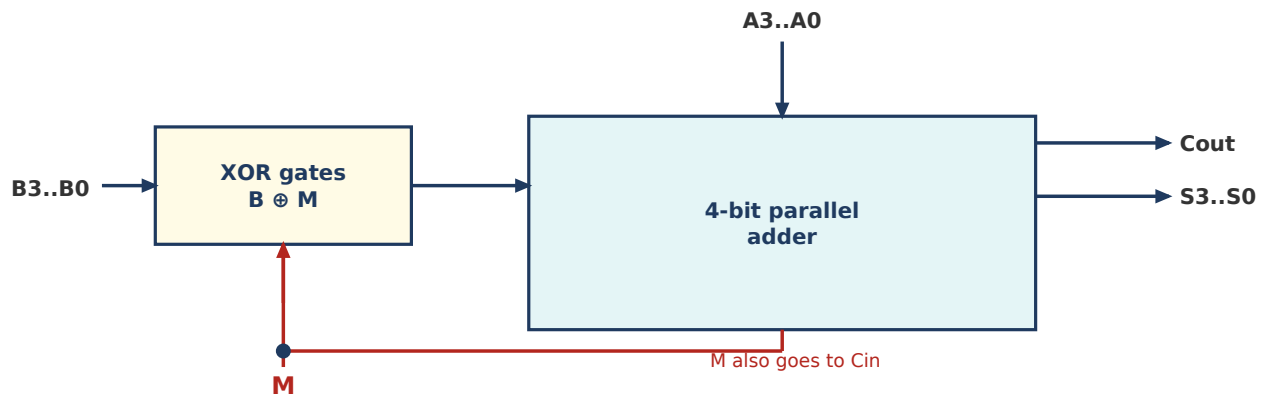
Sum bits $S = P \oplus C$: S0 = 1⊕0 = 1, S1 = 1⊕0 = 1, S2 = 0⊕0 = 0, S3 = 0⊕1 = 1 → **S3S2S1S0 = 1011 = 11**. (6 + 5 = 11 ✓)

	Ripple carry	Carry look-ahead
Speed	Slow (delay grows with n)	Fast (almost the same for any n)
Hardware	Small	Larger (many AND-OR gates, big fan-in)
Carry	Waits for the previous stage	All carries calculated together

9.7 Adder-subtractor circuit

● LEVEL 2 · EXAM

Idea. $A - B = A + (2\text{'s complement of } B) = A + B' + 1$. So we can use the **same adder** for both jobs. A control input **M** decides: $M = 0 \rightarrow \text{add}$, $M = 1 \rightarrow \text{subtract}$.



Each B bit goes through an XOR gate with M: $B \text{ XOR } 0 = B$ (add)
 $B \text{ XOR } 1 = B'$ (subtract)
M also goes into the carry-in (Cin) of the first adder: gives the +1.
Overflow (signed numbers) = $C_{in}(\text{MSB}) \text{ XOR } C_{out}(\text{MSB})$

QUICK TRICK

XOR = controlled inverter. With the XOR gate, $M = 0$ keeps B and $M = 1$ flips B. That is why one XOR per bit is enough. And M also supplies the “+1”, so M does **two jobs**.

Example: $5 - 3$ (4 bits, $M = 1$)

$A = 0101$. $B = 0011$, flipped by XOR $\rightarrow 1100$. $C_{in} = 1$.

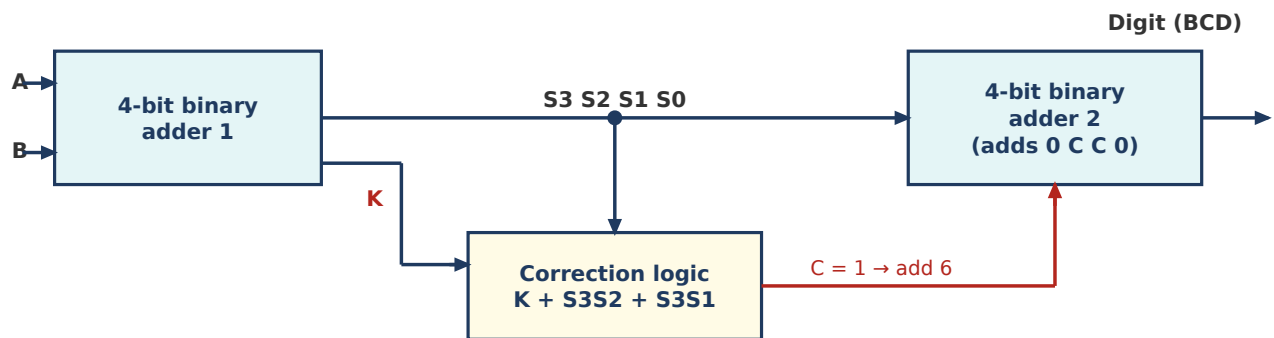
$0101 + 1100 + 1 = 1\ 0010 \rightarrow \text{drop the carry} \rightarrow \mathbf{0010} = \mathbf{2}$. ✓ (Carry out = 1 $\rightarrow$ result is positive.)

9.8 BCD adder

● LEVEL 3 · GATE

Problem. BCD digits (Chapter 2) use only 0000 to 1001. If we add two BCD digits with a normal binary adder, the result can be **more than 9** (invalid in BCD) or can give a carry. **Idea.** If the binary result is bigger than 9, add **6 (0110)** to correct it. (Adding 6 skips the six unused codes 1010–1111.)

- STEP 1 Add the two BCD digits with a 4-bit binary adder.
Call the result $S_3\ S_2\ S_1\ S_0$ and the carry K.
- STEP 2 Is the result bigger than 9? $C = K + S_3.S_2 + S_3.S_1$
- STEP 3 If $C = 1$, add 0110 (6) to the result using a second adder.
 C becomes the carry to the next digit.
- STEP 4 If $C = 0$, the result is already correct.



QUICK TRICK

When to correct: the sum is 10 to 15 (1010–1111) **or** there was a carry out ($K = 1$). $S_3 \cdot S_2 + S_3 \cdot S_1$ catches 1100–1111 and 1010–1011 (any number with $S_3 = 1$ and also $S_2 = 1$ or $S_1 = 1$). Simple rule: **binary sum > 9 → add 6.**

Examples

4 + 3: 0100 + 0011 = 0111 = 7. Not bigger than 9 → no correction → BCD 0111. ✓

8 + 7: 1000 + 0111 = 1111 (15 > 9) → add 0110 → 1 0101 → digits 0001 0101 = **15**. ✓

9 + 9: 1001 + 1001 = 1 0010 ($K = 1$) → add 0110 → 1 1000 → digits 0001 1000 = **18**. ✓

Quick-Solve Tricks — Summary

Circuit	Trick
Half adder	Sum = XOR, Carry = AND. (1 XOR + 1 AND gate)
Full adder	Sum = 1 if odd number of 1s. Cout = 1 if two or more 1s. (2 HA + 1 OR = 5 gates)
Half subtractor	Difference = XOR, Borrow = $A'B$.
Full subtractor	Diff = same XOR as sum. Bout = 1 if $A - B - \text{Bin} < 0$.
Ripple adder	n full adders. Delay $\approx n \times$ carry delay (2 gate delays each).
CLA	$G = AB$ (generate), $P = A \oplus B$ (propagate). $C(i+1) = G + P \cdot C(i)$.
Adder-subtractor	XOR B with M, and use M as Cin. $M = 1$ subtracts.
BCD adder	If binary sum > 9 or carry, add 6 (0110).
NAND-only counts	Half adder 5, full adder 9.

GATE QUICK CHECK — Chapter 9

Half adder : $S = A \text{ xor } B$, $C = AB$
 Half subtractor : $D = A \text{ xor } B$, Borrow = $A'B$
 Full adder : $S = A \text{ xor } B \text{ xor } \text{Cin}$, Cout = $AB + \text{Cin}(A \text{ xor } B)$
 Full subtractor : $D = A \text{ xor } B \text{ xor } \text{Bin}$, Bout = $A'B + A'\text{Bin} + B.\text{Bin}$
 CLA : $G = AB$, $P = A \text{ xor } B$, $C(i+1) = G(i) + P(i)C(i)$
 Adder-subtractor: $B \text{ xor } M$ and Cin = M Overflow = $\text{Cin}(\text{MSB}) \text{ xor } \text{Cout}(\text{MSB})$

Traps: half vs full adder · borrow uses A' · ripple delay grows with n · BCD correction is +6 (not +10) · overflow for signed numbers uses the carries into and out of the MSB.

Practice questions

● LEVEL 1 • BEGINNER

- Q1.** A half adder has $A = 1$, $B = 1$. Find Sum and Carry.
- Q2.** How many full adders are needed for an 8-bit ripple carry adder?
- Q3.** Write the Borrow output of a half subtractor.

● LEVEL 2 • EXAM

- Q4.** Full adder with $A = 1$, $B = 1$, $C_{in} = 1$. Find Sum and Cout.
- Q5.** In a 4-bit ripple carry adder each full adder takes 12 ns to produce its carry. How long until C_4 is ready?
- Q6.** Add $9 + 6$ in BCD.

● LEVEL 3 • GATE

- Q7.** What is the minimum number of 2-input NAND gates for a full adder?
- Q8.** Full subtractor with $A = 0$, $B = 1$, $B_{in} = 1$. Find Diff and Bout.
- Q9.** CLA with $G_0 = 1$, $P_0 = 0$, $G_1 = 0$, $P_1 = 1$, $C_0 = 0$. Find C_1 and C_2 .
- Q10.** 4-bit signed adder: $0111 + 0001$. Find the carry into and out of the MSB, and tell if there is overflow.
- Q11.** A 16-bit ripple carry adder: each full adder has a carry delay of 2 gate delays, and each gate delay is 1 ns. Find the time for the final carry.
- Q12.** BCD adder: add $7 + 5$. Show the steps.

Answers

Q	Solution
1	Sum = $1 \oplus 1 = 0$. Carry = $1 \cdot 1 = 1$. Sum 0, Carry 1 (binary 10).
2	8 full adders.
3	Borrow = A'B.
4	Three 1s (odd) → Sum = 1 . Two or more 1s → Cout = 1 . ($1+1+1 = 11$ in binary.)
5	Given: 4 stages $\times$ 12 ns. Concept: the carry ripples. Answer: 48 ns.
6	$1001 + 0110 = 1111$ ($15 > 9$) → $+ 0110 = 1\ 0101$ → BCD 0001 0101 = 15 .
7	9 NAND gates.
8	Diff = $0 \oplus 1 \oplus 1 = 0$. Real subtraction $0 - 1 - 1 = -2$ (needs borrow) → Bout = 1 .
9	$C_1 = G_0 + P_0 \cdot C_0 = 1 + 0 = 1$. $C_2 = G_1 + P_1 \cdot C_1 = 0 + 1 \cdot 1 = 1$.
10	$0111 + 0001$: the lower 3 bits $111 + 001 = 1000$ → carry into the MSB = 1 . MSB: $0 + 0 + 1 = 1$, no carry out → carry out = 0 . $1 \oplus 0 = 1$ → overflow ($7 + 1 = 8$ does not fit in 4-bit signed).
11	16×2 gate delays $\times$ 1 ns = 32 ns .
12	$0111 + 0101 = 1100$ ($12 > 9$, $S_3 = 1$ and $S_2 = 1$) → correct: $1100 + 0110 = 1\ 0010$ → digits 0001 0010 = 12 .

Next: Chapter 10 — Multiplexer, Demultiplexer, Decoder, Encoder, Comparator and Parity.